

ArkPrism: Evidence-Carrying Source Binding for ArkTS Privacy Analysis

ANONYMOUS AUTHOR(S)

Static taint analysis can track privacy data only after a platform source has been identified and bound to the program value that carries its output. This prerequisite is difficult in OpenHarmony applications. ArkTS APIs are accessed through evolving package exports, factory-returned managers, properties, and compiler-generated aliases, while callbacks and Promise continuations deliver data through values that are absent from a direct-call signature. A missed binding removes the initial taint fact; an unsupported binding introduces flows from unrelated application code.

We present ARKPRISM, which makes *privacy-source binding* an explicit analysis stage. For each source candidate, it resolves $\langle \textit{identity}, \textit{carrier}, \textit{witness} \rangle$: the canonical platform operation, the value that carries privacy data, and the evidence supporting both decisions. Witness-guided resolution combines package exports, receiver origin, access form, SDK declarations, and IR aliases. Continuation-aware IFDS then preserves the bound carrier across ArkTS callbacks, Promise success handlers, and `await`; each reported path retains its source and transfer evidence for inspection.

On a source-first real-project benchmark, ARKPRISM locates all 90 gold source occurrences and rejects all 96 same-name collisions. On the complete 67-case HapBench suite, it reaches 100% precision and specificity and 97.09% F1, improving specificity by 28.57 percentage points and F1 by 2.65 points over HapFlow. A targeted asynchronous benchmark attributes nine additional positive cases to Promise-carrier transfer without changing 24 matched negatives, and a mutation suite classifies all 24 Promise-boundary cases correctly. Manual inspection validates all 90 sampled real-project privacy-data paths. Across 1,014 projects, ARKPRISM identifies 2,339 privacy-API usages and 258 privacy-data paths without resource fallback.

CCS Concepts: • **Security and privacy** → **Software security engineering**; • **Software and its engineering** → **Software defect analysis**.

Additional Key Words and Phrases: OpenHarmony, ArkTS, static analysis, privacy APIs, taint analysis, call-chain analysis

ACM Reference Format:

Anonymous Author(s). 2027. ArkPrism: Evidence-Carrying Source Binding for ArkTS Privacy Analysis. In *Proceedings of The ACM International Conference on the Foundations of Software Engineering (FSE 2027)*. ACM, New York, NY, USA, 29 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnn>

1 Introduction

OpenHarmony has become a major platform for mobile, wearable, and embedded applications, with system services for location, device identity, accounts, media, sensors, networking, and inter-device collaboration [18, 23, 30]. These services make privacy analysis an important part of application review. Static taint analysis addresses this task by identifying values produced by privacy-sensitive APIs and checking whether they can reach logging, storage, network, or other exposure points. Its result, however, is only as reliable as the initial identification of those source values.

This paper studies the prerequisite we call *privacy-source binding*: associating a platform operation with the exact program value that carries its modeled privacy data. Existing ArkTS infrastructure provides strong foundations. ArkAnalyzer constructs typed IR, control-flow graphs, call graphs, and points-to information [11]; HapFlow adds IFDS-based source-to-sink analysis [12]. HapFlow resolves source specifications through SDK method signatures, which supports ordinary direct calls. The difficulty is that a signature alone does not always reveal either the owner of an ArkTS operation or the value produced by it. Once either relation is absent, the downstream solver cannot reconstruct the missing taint seed.

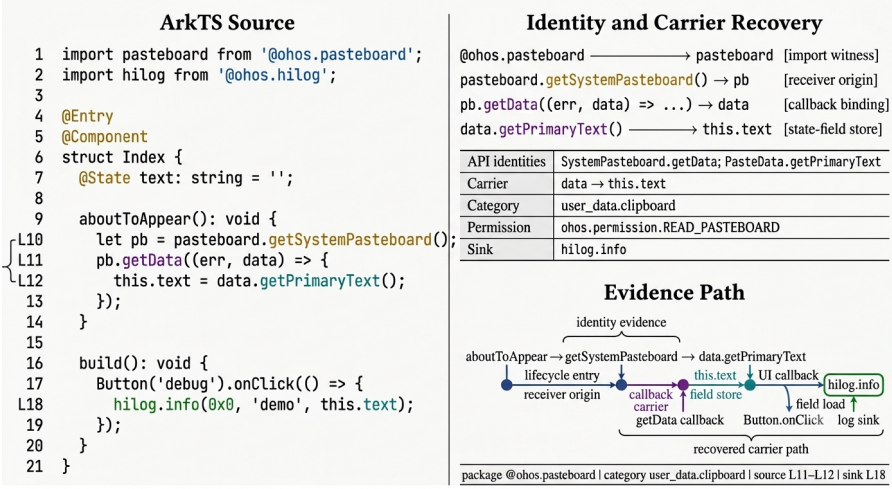


Fig. 1. Privacy-source binding across an indirect ArkTS receiver and callback-carried data. The source identity follows the platform factory, while the carrier follows the callback parameter and component state to the sink.

Two ArkTS characteristics make this problem systematic. First, *API identity is indirect*. The same operation may move from an historical `@ohos.*` module to a consolidated `@kit.*` export, appear as an executable property, pass through an IR temporary, or be invoked on a manager returned by a factory. Matching a final member such as `get`, `create`, or `on` admits unrelated application methods; requiring one exact namespace–method form misses legitimate uses. Second, *the data carrier is discontinuous*. Many APIs deliver data through a callback parameter, a fulfilled Promise value, or an `await` result rather than through the direct call result. Connecting a callback by type alone is also unsafe because a function-valued argument may be stored but never executed.

Figure 1 illustrates both failures. The pasteboard object receives its platform meaning from the factory that created it, rather than from the final member token. The privacy value becomes available inside a callback, crosses a component field, and reaches a later UI event. A method-signature lookup can therefore miss the source owner, the callback-carried value, or both. A broad callback edge can recover the body but still cannot establish that the callback executes with the privacy payload. These are source-binding failures, not limitations of the IFDS propagation algorithm itself.

Our first insight is that no single syntactic feature is a sufficient source-identity test. Package imports are useful for narrowing candidates, receiver types identify some manager calls, factory origins identify others, and executable access form distinguishes properties from same-named constants. ARKPRISM therefore resolves an API only when its access form has a compatible *identity witness*. Common member names require a receiver, call-target, or factory witness; package proximity alone is insufficient. This design preserves valid package migrations and IR aliases without turning imported vocabulary into sources.

Our second insight is that callback execution and privacy-data transfer are separate obligations. An execution witness determines whether a continuation edge exists; a carrier contract determines which value crosses that edge. ARKPRISM records both in an evidence-carrying source binding $\langle \textit{identity}, \textit{carrier}, \textit{witness} \rangle$. It then re-roots the bound carrier at callback parameters and Promise success handlers within IFDS, propagates payload-dependent returns to subsequent continuations, and rejects rejection handlers, unrelated Promise owners, and constant-return transforms. The

same representation covers direct returns and `await` results without introducing construct-specific source identities.

We implement these insights in ARKPRISM, an ArkTS privacy analyzer built on ArkAnalyzer and HapFlow. Its pipeline first resolves source bindings from package exports, SDK declarations, receiver origin, access form, and ArkIR. It then propagates the bound values through initializers, closures, exceptions, access paths, callbacks, Promise chains, and lifecycle-aware entries. The output is an evidence graph that links each accepted source and carrier to its call context, sink, transfer derivation, and source location. This graph is the audit record of the two analyses rather than a separate source of inferred behavior.

The evaluation aligns each claim with an independent oracle. A source-first real-project benchmark contains 90 gold source occurrences and 96 same-name collisions; ARKPRISM classifies all 186 at exact locations. On all 67 HapBench cases, ARKPRISM obtains 100% precision and specificity and 97.09% F1, improving specificity by 28.57 percentage points and F1 by 2.65 points over HapFlow. ArkAsyncBench isolates continuation transfer: removing Promise success-handler binding loses nine positive cases, reducing recall from 100% to 62.50% ($p = 0.00390625$), while all 24 matched negatives remain unchanged. A mutation-based Promise suite tests ownership, execution, handler position, alias separation, value dependence, sequential continuations, and flattening; ARKPRISM classifies all 24 cases correctly. Manual source/IR review confirms all 90 sampled real-project privacy-data paths. Finally, ARKPRISM completes 1,014 projects without resource fallback and reports 2,339 privacy-API usages and 258 privacy-data paths.

This paper makes three contributions:

- **Privacy-source binding as an analysis problem.** We identify API-owner recovery and data-carrier recovery as coupled prerequisites for ArkTS taint analysis, and represent their result as an inspectable $\langle \textit{identity}, \textit{carrier}, \textit{witness} \rangle$ binding.
- **Witness-guided identity and continuation analysis.** We combine package/export, receiver-origin, access-form, SDK, and IR evidence with carrier-aware IFDS transfer for callbacks, Promise success continuations, and `await`. The analysis distinguishes callback execution from payload transfer and retains the derivation of every reported path.
- **Multi-level empirical evidence.** We evaluate exact-location source binding, end-to-end flow classification against HapFlow, asynchronous semantic boundaries, manually reviewed real-project paths, and scalability over 1,014 projects.

2 Background

2.1 ArkTS Programs and Platform APIs

OpenHarmony applications are organized around abilities, pages, ArkUI components, lifecycle methods, and event callbacks [19, 30]. ArkTS retains TypeScript-like classes, functions, fields, closures, and Promises while adding a declarative UI model. A component's `build` method describes UI structure; methods such as `aboutToAppear` and handlers such as `onClick` are invoked by the framework. Privacy-relevant behavior can consequently span an initializer, a framework callback, component state, and a later event even when no explicit source-level call connects these methods.

ArkAnalyzer lowers ArkTS into ArkIR and represents anonymous functions, static and instance initialization, closures, exception flow, and access paths in generated or transformed methods [11]. These methods remain semantically relevant: they contain callback assignments, factory calls, captured values, and component initialization. Our analysis uses ArkAnalyzer's Scene, typed IR, call graph, and pointer evidence rather than interpreting ArkTS text in isolation.

Platform APIs add a second form of indirection. OpenHarmony has evolved from fine-grained `@ohos.*` modules toward consolidated `@kit.*` exports, and applications may mix both forms. An

imported namespace can be renamed, assigned to an IR temporary, or used to create a typed manager on which the privacy operation is invoked. Some configured operations are executable property reads rather than calls. The analysis must therefore preserve API ownership across source syntax, SDK declarations, and ArkIR.

2.2 Taint Analysis and Source Binding

An interprocedural taint analysis starts from a set of source facts and propagates them through program statements until they reach configured sinks. IFDS represents this computation over an exploded interprocedural supergraph and supports precise, distributive flow functions [35, 36]. HapFlow instantiates this approach for ArkTS and models closures, lifecycle entries, and source-sink queries on top of ArkAnalyzer [12].

Before propagation begins, each source specification must answer two questions. The *identity* question determines whether a program operation denotes the configured platform API. The *carrier* question determines which program value receives the modeled data. For a direct-return API, the carrier is normally the assigned result. For a callback API, it is a designated callback parameter. For a Promise API, it is the fulfilled value observed by a success continuation and, after `await`, the assigned result. An operation may be privacy-relevant without producing modeled data; such an occurrence remains detector evidence but does not create an IFDS seed.

We use *source binding* for the accepted association between an API identity and its carrier. A source binding also retains a witness explaining why the owner and carrier were accepted. This evidence is important in ArkTS because package presence, member spelling, function type, and control-flow reachability are individually insufficient. Section 3 defines the binding and shows how its witness is preserved during propagation.

2.3 Scope of the Analysis

ARKPRISM answers two audit questions: which configured privacy-sensitive platform operations occur, and which modeled values may reach configured sinks. It does not equate an API occurrence or a may-flow with a policy violation. The detector and taint query consequently remain separate. Detector rules cover configured calls and executable properties; query rules additionally require a concrete return, callback, argument, or framework-input carrier. Detector-local sink observations provide nearby review context, whereas configured-query paths are produced by IFDS or explicitly labeled continuation recovery. Keeping these evidence sets distinct avoids interpreting every API occurrence as a leak or every nearby sink as a data-flow endpoint.

3 Design of ARKPRISM

ARKPRISM addresses privacy-source binding in two stages. It first resolves which platform operation a statement denotes and which value, if any, carries the modeled privacy data. It then propagates that bound value while preserving the evidence used to create it. ArkAnalyzer supplies typed ArkIR, call graphs, and pointer information [11]; HapFlow supplies the IFDS substrate [12]. ARKPRISM adds the source-binding and continuation semantics that connect these layers.

3.1 Architecture Overview

Figure 2 gives the end-to-end architecture. ArkAnalyzer transforms each project into a Scene, ArkIR, and call graph; the analysis specification provides source, carrier, package-migration, and sink rules. ARKPRISM first resolves an evidence-carrying source binding, then propagates its carrier through the IR and witnessed asynchronous continuations. Lifecycle ordering supplies execution context, and evidence assembly attaches the retained source and transfer witnesses to JSON and DOT reports.

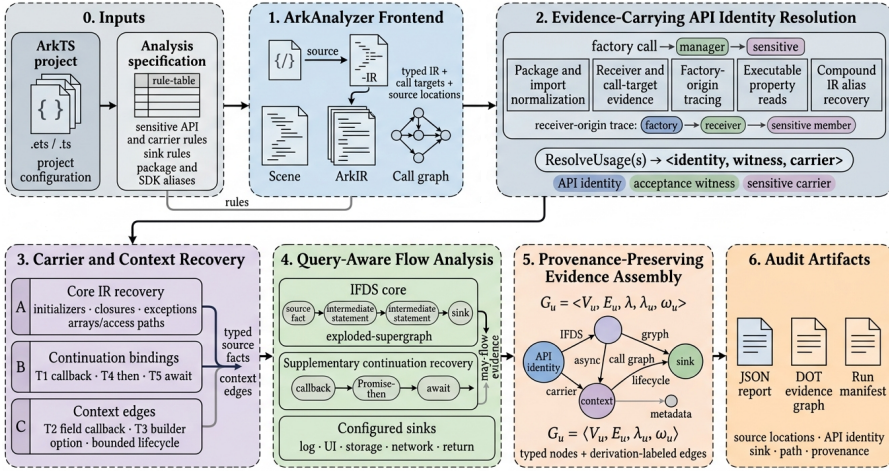


Fig. 2. ARKPRISM analysis pipeline and evidence assembly.

3.2 Privacy-Source Binding

ARKPRISM accepts two related rule sets. Detector rules R_D identify configured platform operations by package, namespace, member, access form, optional receiver factory, permission, and privacy category. Query rules R_T additionally name an SDK owner and a concrete data carrier: a return value, an argument, or a designated callback parameter. Query rules also distinguish privacy data from explicitly modeled framework inputs. A detector rule can therefore describe a privacy-relevant operation without asserting that the operation produces a taint source.

For a candidate statement s and a compatible rule r , the analysis returns one unit:

$$\text{ResolveUsage}(s, r) = \langle \text{identity}, \text{carrier}, \text{witness} \rangle. \quad (1)$$

The identity records the normalized package/namespace, receiver owner, member, and access form. The carrier is the assigned result, a designated argument or callback parameter, or $\perp$ for detector-only evidence. The witness is the strongest compatible ownership evidence available at the statement: an imported namespace, an IR alias, an exact SDK type or call target, a traced factory origin, or an executable property read. We write $\text{Id}(s)$, $\text{Carrier}(s, r_T)$, and $w(s, r)$ for these three components. The accepted detector occurrences form U_D .

The binding must satisfy two independent conditions. First, the member and access form must agree with the rule and carry an access-appropriate identity witness. Second, a query source must name a concrete carrier that is valid for the resolved SDK declaration. Return sources require a non-void result; callback sources require a typed callback and valid callback/parameter positions; argument sources are admitted only as explicit framework inputs. The loader rejects a query rule that cannot establish this carrier contract.

For a direct return, the carrier becomes an IFDS fact at the assignment. For a callback source, the carrier becomes a fact at the designated parameter of the resolved callback edge. Promise transfer preserves the same source witness while re-rooting the carrier from the Promise object to the success-handler parameter and, when the handler returns a payload-dependent value, to the next Promise. Callback edges recovered only for call context remain distinct and cannot create a taint fact.

Each accepted occurrence and path retains this binding. The report graph links API, caller, sink, and taint-path nodes, while edge metadata records whether a relation came from the call

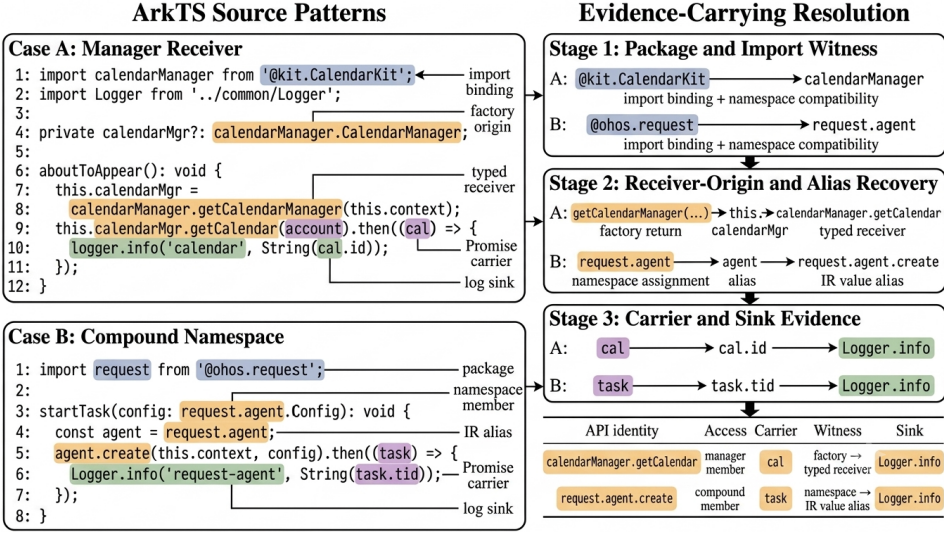


Fig. 3. Evidence-carrying resolution for manager receivers and compound namespaces.

graph, detector-local tracing, IFDS, or bounded supplementary recovery. Provenance identifies the producing analysis and transfer rule; it is not treated as a confidence score. This design makes the report an inspectable derivation of source binding and propagation rather than an additional inference layer.

3.3 Running Example: Why Direct Calls Are Insufficient

Figure 3 shows an anonymized pattern distilled from manually reviewed benchmark projects. The example is intentionally small, but each highlighted source position corresponds to a real source-level evidence class seen during manual annotation: a manager object returned by a Harmony API, a member call on that manager, a compound namespace API that may be lowered through a temporary, and a local sink that helps an auditor inspect the behavior.

A direct-call baseline that only matches `namespace.method(...)` statements can recover neither `this.calendarMgr.getCalendar(...)` nor `agent.create(...)` reliably. The former is a member call whose receiver obtains its privacy meaning from `calendarManager.getCalendarManager`; the latter is a separate compound-namespace example whose `request.agent` prefix may be assigned to an IR temporary before `create` is invoked. ARKPRISM treats both cases as rule- and evidence-matching problems over imports, aliases, receiver origins, and IR-lowered expressions, then retains lifecycle context, local sink evidence, configured-query paths, and source metadata instead of returning flat method-name hits.

3.4 Namespace-constrained Package Migration

OpenHarmony API evolution makes exact package matching brittle, but consolidated kit packages are not globally equivalent to their historical modules. We therefore maintain a directional migration relation $\mathcal{M}$ over package-namespace pairs. For example, the `deviceInfo` export maps from `@ohos.deviceInfo` to `@kit.BasicServicesKit`, while unrelated exports in the same kit do not become compatible. A detector rule r_D and import binding i are compatible when they agree exactly or when $\mathcal{M}$ contains the documented export migration:

$$Compat(i, r_D) \triangleq (i.pkg, i.ns) = (r_D.pkg, r_D.ns) \vee ((i.pkg, i.ns), (r_D.pkg, r_D.ns)) \in \mathcal{M}. \quad (2)$$

The analyzer also derives namespace aliases from import declarations. For import `wifi` from `@ohos.wifiManager`, it records `wifi` as an alias of the canonical `wifiManager` namespace. Kit imports retain both the textual binding and the exported namespace. Matching uses the normalized package–namespace pair and member, while the report retains the raw import and SDK signature for audit.

3.5 Package- and Receiver-aware API Identity Resolution

HapFlow’s namespace–method loader resolves configured entries to SDK declarations, but its schema does not encode executable property reads or retain factory-origin evidence for manager/helper receivers. ARKPRISM represents these access forms by computing a semantic identity and an acceptance witness for each candidate usage.

3.5.1 Witness-guided Acceptance. Given a candidate statement s , ARKPRISM computes its member, access form, and receiver origin. Package compatibility and member agreement generate a candidate; an access-specific witness decides acceptance:

$$Accept(s, r_D) \iff CompatPkgNs \wedge MemberMatch \wedge ModeMatch \wedge SufficientWitness. \quad (3)$$

Namespace calls require an imported namespace or recovered IR alias. Property rules require an executable field read. Manager calls require an exact SDK receiver type, resolved declaring-class target, traced factory origin, or exact namespace receiver. The last option is disabled for generic members such as `get`, `on`, `start`, `request`, and `close`; these members require type, target, or factory evidence. Package presence or final-token agreement alone is never sufficient.

IFDS source-rule acceptance. The IFDS query uses an exact SDK method signature whenever ArkAnalyzer resolves the call target. If the target remains unknown after frontend recovery, the fallback must match member, arity, normalized parameter types, owner evidence for generic members, and any string-dispatched event literal:

$$Accept_T(s, r_T) \triangleq Exact_T(s, r_T) \vee (CompatibleFallback_T(s, r_T) \wedge OwnerGuard \wedge LiteralGuard). \quad (4)$$

Exact arity is required whenever the rule declares a parameter array, including an empty one. Parameter-type comparison normalizes imported qualifiers, whitespace, array notation, callback aliases, and union order. Parameter names remain diagnostic metadata because they are not part of call semantics. Consequently, `avPlayer.on('error', ...)` cannot match a configured geolocation or sensor event merely because both use `on` with the same arity. This stricter predicate governs source seeding; detector acceptance remains independent.

3.5.2 Receiver-Origin Tracing. The key addition beyond namespace–method loading is $Recv(s)$. For a statement $s : x.m(\dots)$ where x is a local or field variable, ARKPRISM traces x backward through assignments and instance-invoke bases to find its origin, subject to a tracing bound of k_{recv} steps (default 8) to avoid unbounded backward exploration:

$$Recv(s) = \begin{cases} Ns(def), & x \leftarrow invoke(q.f()), f \in factory(r_D), \\ Recv(def), & x \leftarrow y, y \text{ has known origin,} \\ TF(x), & type(x) = T, T \in SDK, \\ \perp, & \text{unresolvable in } k_{recv} \text{ steps.} \end{cases} \quad (5)$$

Table 1. API identity resolution examples.

Source pattern	Pkg. family	Ns./recv.	Origin	Member	Acc.	Acceptance witness
<code>id.getOAID(cb)</code>	BasicSvc	<code>id</code>	$\perp$	<code>getOAID</code>	direct	Import + namespace
<code>net=getDefaultNet()</code>	NetworkKit	<code>net</code>	$\perp$	<code>getDefaultNet</code>	assign	Import + assignment
<code>mgr.getData(cb)</code>	BasicSvc	<code>mgr</code>	<code>getMgr</code>	<code>getData</code>	indirect	Factory return
<code>deviceInfo.brand</code>	BasicSvc	<code>devInfo</code>	$\perp$	<code>brand</code>	property	Executable read

The first case covers the common pattern $mgr \leftarrow getManager(ctx); mgr.getData(cb)$, where the receiver *mgr* originates from a privacy API factory call. The second case propagates origin through intra-procedural assignment chains and instance-invoke bases. The third case (*TF*) uses an exactly compatible declared SDK type. Inter-method field assignments are not claimed as origin-proven unless the defining statement or an exact type/target identity is recovered; otherwise the call is rejected. This rule avoids silently treating package import plus method-name agreement as a typed call target.

3.5.3 Access-Kind Classification. Each recognized usage receives an *Acc* label:

- *direct*: bare invoke statement, e.g., `id.getOAID(cb)`;
- *assign*: invoke with return value assigned, e.g., `net = getDefaultNet()`;
- *indirect*: invoke on a traced receiver, e.g., `mgr.getData(cb)` where $Recv(mgr) \neq \perp$;
- *property*: field/property read, e.g., `deviceInfo.productModel`.

Table 1 illustrates how *Id* resolves each access kind.

IR-lowered multi-segment APIs require an additional alias relation A_{ir} . For assignments of the form $t := q.member$, ARKPRISM records $(t, q.member) \in A_{ir}$. A later statement $t.create(\dots)$ is matched as $q.member.create(\dots)$ if the composed signature appears in R_D . This is essential for APIs such as `request.agent.create`, whose namespace can disappear after lowering.

3.6 Lifecycle Context

OpenHarmony applications follow a framework-managed lifecycle. A *UIAbility* moves through creation, foreground, background, and destruction; components move through appearance, build, visibility, and disappearance; UI handlers execute after registration. These transitions supply execution context for source bindings but do not alter their identity or carrier.

ARKPRISM represents ability phases, component phases, and the cross-layer transitions between them. Cross-layer edges include relations such as `onForeground` to `aboutToAppear`. This explicit ordering connects source production and later consumption without treating every lifecycle method as mutually reachable.

The lifecycle modeler discovers ability classes by checking superclass inheritance from *UIAbility*, *Ability*, and extension ability base classes; component classes by checking superclass (*CustomComponent*, *ViewPU*), decorators (*@Component*), or the presence of two or more component lifecycle methods; and callback methods by scanning IR statements for *CommonMethod.** invocations and extracting *FunctionType* arguments.

Lifecycle information is consumed in two distinct ways. For call-chain recovery, each transition $(m_i, m_j) \in \delta_a \cup \delta_c \cup \delta_{cross}$ becomes a provenance-labeled implicit edge. For IFDS, the model orders entry methods in a synthetic *DummyMain*: ability creation and foreground callbacks precede component initialization and UI callbacks, followed by background and destruction phases. The bounded default removes the standard back edge from the final entry to the loop header, preventing arbitrary destruction-to-creation cycles. Repeatable event classes may receive one additional explicit

Table 2. Asynchronous carrier and context recovery rules.

Rule	Construct	Recovery relation	Required witness	Domain
T1	Callback source	$Resolve_T(a_i) \rightarrow cb.param_j$	Typed callback and carrier index	IFDS call flow
T2	Field callback	$Read(this.f) \rightarrow cb$	Traced field assignment	Report CG
T3	Builder option	$Arg(bld, f) \rightarrow cb$	Named option field	Report CG
T4	Promise then	$fact(base) \rightarrow cb.param_0$	Success-handler dependence	IFDS call flow
T5	async/await	$fact(promise) \rightarrow await_{res}$	Explicit await expression	Native IFDS normal flow

firing, but no unbounded loop. The IFDS solver derives interprocedural targets from the Scene and pointer information; it does not consume the separately augmented report call graph.

3.7 Asynchronous Carrier and Context Recovery

OpenHarmony privacy APIs often deliver sensitive data through asynchronous callbacks and Promise chains rather than direct return values. For example, `geoLocationManager.getCurrentLocation(callback)` passes location data to a callback parameter, while `wifiManager.getLinkedInfo().then(callback)` delivers Wi-Fi state through a Promise resolution. Standard call-to-return propagation does not by itself establish the connection between a source invocation and a callback parameter or Promise resolution, so these constructs require explicit transfer rules.

ARKPRISM formalizes carrier binding as $SrcBind(call, cb, param)$ (Section 3.2). The five rules occupy two analysis domains. T1 and T4 add construct-specific IFDS call flow, T5 follows native normal flow over the explicit ArkIR `await` use, and T2–T3 recover callback context for the report graph without manufacturing taint facts. Table 2 makes this distinction explicit.

3.7.1 Callback ICFG Edges and Fact Domain. ArkAnalyzer represents an arrow function argument with a `FunctionType` carrying its generated method signature; `ClosureType` additionally records the lexical environment. ARKPRISM applies this typed lookup to every statement containing an invocation, including assignments such as `q = p.then(cb)`. A function type alone does not establish execution. The ICFG admits a callback edge only when one of four witnesses is present: a configured callback-source contract, a lifecycle-governed ArkUI event, a registered platform task with the expected owner and callback position, or the fulfillment handler of a witnessed Promise owner.

Promise ownership is established by a Promise receiver or target, a configured `Promise<T>` source, or a receiver-qualified SDK declaration whose arity-compatible overloads all return a Promise. Callback-returning overloads and resolved application methods named `then` do not satisfy this condition. ArkUI and platform-task edges expose executable callback context but carry no privacy payload unless a source binding independently supplies one. Other function-valued registrations remain report context and are not converted into execution.

An evidence-carrying fact is

$$d = \langle v, \alpha, \kappa, \epsilon, \delta \rangle, \quad (6)$$

where v is the root ArkIR value, α is its field or array access path, ϵ is immutable source evidence, δ is transfer metadata, and κ distinguishes a direct value, Promise payload, callback payload, or framework argument. Fact identity includes $\langle v, \alpha, \epsilon, \kappa \rangle$; transfer metadata does not create a new semantic fact. Promise binding uses

$$ReRoot(d, v', \kappa') = \langle v', \emptyset, \kappa', \epsilon(d), \delta(d) \rangle, \quad (7)$$

which clears the container's access path while preserving its source witness and accumulated derivations.

3.7.2 T1: Callback Parameter Binding. When an R_T source accepts a callback argument at position i , T1 resolves the compiler-provided function signature to a callback method with a CFG and binds only the payload parameter selected by the rule. Because `ClosureType` refines `FunctionType`, this lookup also preserves the lexical environment. Definition-backed recovery remains available only in the separately labeled report/supplementary analysis; it cannot manufacture an IFDS edge. For `getData((err, data) ⇒ ...)`, T1 binds `data`, not every callback parameter.

The tracer seeds only $param_j$ selected by $r_T.i_{src}$, not every callback parameter. It propagates through ArkIR value-use edges and assignments, and accepts a sink only when a sink argument structurally depends on the current fact. IR-local names are never compared by substring, and control dependence alone does not constitute an explicit data-flow edge.

3.7.3 T2: Field-Stored Callback Binding. ArkUI components frequently store callback functions in class fields (e.g., `this.handler = () => {...}`). The callback is registered with a framework API at a later point, creating a temporal gap between definition and invocation. ARKPRISM reads the field value at the registration site and traces it to the assignment, adding $Read(this.f) \rightarrow cb$ to the report call graph. This handles the pattern where `this.onSelect` is passed to `List.onSelect(this.onSelect)` in a component's build method. T2 supplies entry/call context; it does not by itself assert a sensitive carrier or inject an IFDS fact.

3.7.4 T3: Builder-Option Callback Binding. ArkUI builder APIs accept configuration objects whose fields are callback functions (e.g., `bindPopup({builder: popupBuilder})`). T3 extracts the callback from a named option field, $Arg(bld, f) \rightarrow cb$, and adds the recovered owner-builder edge to the report call graph. The pattern is structurally distinct from T1 because the callback is nested inside a lowered object initializer rather than passed as a positional source callback. As with T2, the edge provides call context and is not an IFDS seed.

3.7.5 T4: Promise Success-Continuation Binding. `Promise .then()` carries the receiver's resolved value into the success handler's first data parameter. For call site $s : b.then(a_0)$, resolved callback $cb = Resolve(a_0)$, and incoming fact d , ARKPRISM extends the IFDS call-flow function with

$$\mathcal{F}_{call}^{then}(s, cb, d) = \begin{cases} \{ReRoot(d, cb.param_0, CallbackPayload)\}, & PromiseGuard(s, d) \wedge Root(b, d), \\ \emptyset, & \text{otherwise.} \end{cases} \quad (8)$$

Here $Root(b, d)$ requires $\alpha(d) = \emptyset$ and exact value identity or an available pointer alias between b and $v(d)$. A property of a `Promise` object is therefore not rebound as its resolved payload. $PromiseOwner(s)$ proves that the success callback may execute; it does not itself create sensitive data. $PromiseGuard$ separately requires $\kappa(d) = PromisePayload$, an immutable source witness whose carrier is `Promise<T>`, and the owner evidence above. Consequently, a non-sensitive SDK `Promise` may expose a callback body containing a later source, but its resolved value is not re-classified as privacy data. The re-rooted fact preserves $\epsilon(d)$ and records `PROMISETHEN`. Rejection handlers and unrelated callback positions receive no fact.

For an assigned continuation $s : q = b.then(cb)$, taint reaches q only through a tainted callback return:

$$\mathcal{F}_{ret}^{then}(return\ x, s, d) = \begin{cases} \{ReRoot(d, q, PromisePayload)\}, & Returned(x, d), \\ \emptyset, & \text{otherwise.} \end{cases} \quad (9)$$

Returned(x, d) holds when normal flow has produced a fact whose structured value matches the return operand x ; thus an alias or property-derived value can match with its access path intact, whereas mere control dependence or consumption by a constant-returning transform cannot. This return-flow rule records PROMISERETURN, supports sequential continuations and Promise flattening, and rejects a chain whose first handler returns an independent constant.

3.7.6 T5: Native Async/Await Normal Flow. For `data = await promiseMethod()`, ArkIR exposes a use from the resolved Promise value to `data`. The native normal-flow function therefore maps the Promise payload to the assignment target and its later uses; no continuation-specific call-flow rule is required. A bounded fallback handles wrapper Promises whose executor-to-await relation is absent from ArkIR, and labels those outcomes as supplementary rather than IFDS-derived.

3.7.7 Algorithm. Algorithm 1 gives the T4 extension to IFDS call flow. It executes on an edge in E_{then} , not after solver termination. Promise-payload facts bypass positional argument mapping: the resolved value is not an ordinary argument of `then`. Non-payload facts may still enter through exact lexical captures. The transfer is monotone and adds a source-preserving fact only when Promise ownership, payload state, root identity, and callback-position evidence hold.

Algorithm 1 Promise success-continuation IFDS call flow

Require: Call site s , resolved callee m , incoming fact d
Ensure: Call-flow facts D'

```

1: if  $(s, m) \notin E_{then}$  then
2:   return STANDARDCALLFLOW( $s, m, d$ )
3: end if
4: if  $d = 0$  then
5:   return  $\{0\}$ 
6: end if
7: if  $\kappa(d) \neq \text{PromisePayload}$  then
8:   return CAPTUREDCALLFLOW( $s, m, d$ )
9: end if
10:  $D' \leftarrow \emptyset$ 
11: if PROMISEGUARD( $s, d$ )  $\wedge$  ROOT( $s.base, d$ ) then
12:    $d' \leftarrow \text{REROOT}(d, m.param_0, \text{CallbackPayload})$ 
13:    $d'.derivation \leftarrow d'.derivation \cup \{\text{PROMISETHEN}\}$ 
14:    $D' \leftarrow D' \cup \{d'\}$ 
15: end if
16: return  $D'$ 
```

Relative to the resolved callback edge and available alias relation, this transfer follows Promise success semantics: every accepted fact denotes a modeled Promise payload that may be supplied as the first success-handler argument. The guard excludes catch, rejection callbacks, nonzero callback positions, custom resolved then owners, and receiver facts without Promise-payload state. The source witness is copied rather than reconstructed, so source-distinct IFDS facts remain distinct across the continuation.

Figure 4 makes the transfer boundary concrete. Green dependence reaches a success parameter, a same-Promise alias, or a payload-dependent continuation return; red dependence is rejected at handler position, Promise ownership, rejection operators, or a payload-independent return. These cases distinguish continuation semantics from matching a member named `then`.

3.8 Callback-augmented Call-chain Recovery

For a recognized usage, call-chain recovery searches backward over ArkAnalyzer call edges, IR-level invocations missing from the graph, witnessed callback edges, and links from generated

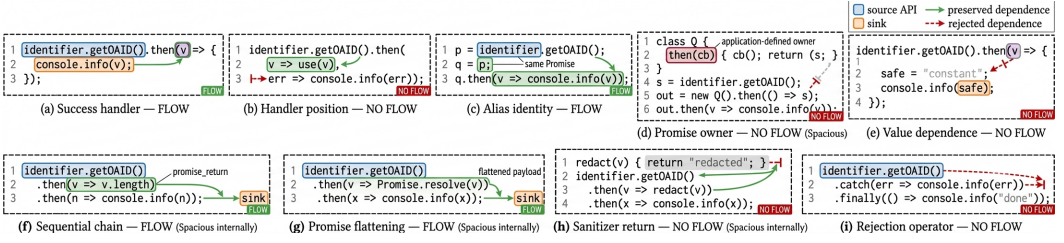


Fig. 4. Promise ownership and payload-dependence boundaries.

initialization methods to their owning class or component. The search stops at a framework entry, an initializer, or the first method for which no justified predecessor exists. Each chain is labeled accordingly as framework, initialization, local, or *unk*. A local chain confirms the containing method but is not reported as a recovered framework-entry path.

3.9 Sink and Taint Attachment

For each detector occurrence $u \in U_D$, local tracing extracts statements $K_D(u)$ that may expose privacy-related data through logging, UI display, storage, network transfer, or data return. Sink resolution requires an exact configured method token and compatible receiver/namespace evidence; in particular, substring agreement such as `set` versus `setData` is rejected.

The IFDS query is configured independently by R_T and its sink set. A result $q = \langle s, k, \rho \rangle$ states that the carrier bound at source s may reach sink k along path ρ . Every fact retains the source kind, API owner, carrier, SDK signature, rule origin, and normalized source location. Normal, call, continuation, return, and exceptional transfer copy this source witness unchanged, so facts from different source bindings remain distinct even when they reach the same statement.

The report distinguishes paths produced by IFDS, bounded asynchronous recovery, or both. It separately records whether Promise-success and Promise-return transfer contributed to an IFDS path. Privacy-data and framework-input paths also remain separate. A path is linked to a detector occurrence only when project, normalized location, and API identity agree; framework-input paths are never attached to privacy-API occurrences. Unlinked paths remain visible as project-level query evidence instead of being forced onto an unrelated detector record.

Construction invariants. Three invariants connect the design to the implementation. *Well-formed seeding* requires every path to originate from one valid carrier and source kind. *Witness preservation* keeps the original source evidence unchanged across all transfer functions. *Link safety* attaches only privacy-data paths whose project, source location, and API identity agree with the detector occurrence. The loader, fact keys, transfer functions, and evidence linker enforce these properties; the evaluation checks their observable consequences through rule-resolution audits, endpoint integrity, and manual path review.

3.9.1 Provenance Annotation. The report stores provenance where it is produced. A call-chain edge records its call type; a query path records its producer and continuation derivations; an endpoint link records exact-statement or normalized-location agreement. Evidence assembly retains these fields instead of replacing them with a global confidence score. An asynchronous supplement therefore cannot be presented as an IFDS path, and Promise-return transfer remains distinguishable from initial Promise binding.

3.10 Analysis Contract

ARKPRISM’s recognition claim is parameterized by ArkAnalyzer IR, the fixed API rule set, and the identity witness required by *Accept*. The analysis targets statically represented ArkTS code; native code, dynamic loading, runtime dispatch unavailable to the frontend, and APIs absent from the rule set are outside the query domain. A recognized API establishes an executable use under this contract, while policy-level violation assessment remains a downstream audit decision.

Configured-query results are may-flows under their source/sink and reachability models. The evidence model preserves the producing transfer and admits a detector occurrence as an IFDS source only through a source-endpoint link. The solver distinguishes complete, resource-bounded, and failed analyses; only complete analyses contribute negative evidence. Edge deduplication is scoped by IR statement, value identity, access path, and source witness because same-named locals in different methods and facts from different seeds remain distinct. HapBench positive and negative cases exercise these invariants, and project isolation enforces them at corpus scale.

3.11 Scalable Execution

SDK declarations are loaded lazily into the Scene when taint analysis starts, keeping frontend construction independent of query-specific augmentation. Source and sink rules resolve against the designated SDK, and IFDS resource bounds remain explicit. Project isolation prevents cross-project frontend state, while phase-aware evidence retention excludes incomplete analyses from negative results.

Let C be the number of executable call/property candidates, d the maximum number of package/namespace-compatible rules returned by the rule index, and k_{recv} the receiver-definition bound. Identity resolution costs $O(Cdk_{recv})$ time and $O(C)$ output space; in practice d is small because package and namespace indexing precede member matching. The evidence carried by a source fact is immutable and constant-sized. T4 is evaluated only on resolved callback call edges and adds at most one fact per incoming source fact; its additional work is constant apart from the already available alias query. The optional unresolved-IR fallback is independently capped by source candidates, visited methods and values, emitted outcomes, and path length, and every retained fallback outcome remains labeled as supplementary.

The complete pipeline follows a fixed order: Scene construction, lifecycle modeling, report call-graph construction, import/alias extraction, API identity resolution via *Accept*, callback-augmented chain recovery, detector-local sink tracing, continuation-aware IFDS analysis from the lifecycle-aware DummyMain, optional unresolved-IR recovery, endpoint linkage, and evidence-graph assembly. A key property is that ARKPRISM never treats a missing layer as proof of absence: if a call-chain predecessor is absent, the usage still becomes a local evidence chain; if a taint path has no compatible detector endpoint, both records remain visible but unlinked.

4 Implementation

ARKPRISM is implemented in TypeScript as a command-line analyzer. It reuses ArkAnalyzer’s Scene, ArkIR, call graph, and pointer-analysis interfaces, and extends HapFlow’s IFDS client with source-binding and continuation transfer. The analyzer processes one project per isolated frontend Scene and emits machine-readable JSON together with optional DOT evidence graphs.

4.1 Versioned Source Specifications

The detector configuration contains 716 records across 58 system-package groups: 453 namespace calls, 213 manager/receiver operations, and 50 executable properties. The records represent 710 unique package–namespace–member identities, 38 privacy categories, and 74 permission labels.

Each record stores its access form and, where applicable, the factories that establish receiver ownership. A namespace-constrained export map relates historical modules to their corresponding `@kit.*` exports without treating an entire kit as one namespace.

The taint query is deliberately smaller. It contains 257 privacy-data rules whose API-20 declarations expose a concrete return or callback carrier, plus two explicitly modeled framework-input rules. Each rule records source kind, SDK owner, signature metadata, carrier form, and callback/parameter positions when required. Structural validation rejects missing source kinds, ambiguous owners, non-concrete return types, malformed callback positions, and unresolved carrier types. All 257 privacy-data rules resolve uniquely under API-20; 249 also resolve under the API-17 SDK used by HapBench, with the remaining eight retained as explicit version differences rather than redirected to same-named declarations.

4.2 Source-Binding Engine

The detector indexes rules by package and namespace before scanning executable ArkIR statements. It normalizes import aliases and documented exports, classifies each candidate as namespace call, assigned call, manager call, or property read, and recovers compound namespaces when lowering assigns a prefix such as `request.agent` to a temporary. For manager calls, a backward definition walk of at most eight steps searches assignments, factory invocations, exact SDK types, and resolved call targets. The selected ownership evidence is stored with the occurrence. Package-only and member-only candidates are rejected.

The IFDS source loader first uses exact SDK targets. An unresolved target can match only when member, arity, normalized parameter types, owner evidence, and any event literal agree with the rule. This prevents generic members such as `on`, `get`, and `create` from inheriting an unrelated source identity. Source creation then follows the rule's carrier contract: direct results seed the assigned value, callback sources seed only the designated payload parameter, and explicit framework inputs seed their specified argument.

4.3 Continuation-Aware IFDS

The solver derives interprocedural targets from the Scene and pointer information. It resolves function-valued arguments on invocation statements, including assigned Promise calls, but creates an executable callback edge only for a configured callback source, a known ArkUI event, a witnessed platform-task contract, or a witnessed Promise fulfillment handler. Platform-task contracts constrain the API owner and callback position. Promise contracts additionally distinguish Promise-returning overloads from callback overloads and reject application-defined methods named `then`.

IFDS facts include the structured ArkIR value and access path, carrier state, and immutable source binding. At a Promise success edge, the transfer requires a Promise-payload fact rooted at the receiver and re-roots it to the first success-handler parameter. Return flow re-roots only values that depend on the incoming payload, which supports sequential continuations and Promise flattening while stopping constant-return transforms. Rejection handlers, unrelated aliases, and non-Promise owners receive no payload fact. Native ArkIR use edges handle `await`; a separately labeled bounded analysis is used only when callback or Promise relations are absent from ArkIR.

The implementation preserves initializers, closure environments, exceptional successors, constant-index access paths, and source-distinct fact identity. It analyzes all sources jointly; batching is available only as an explicit resource fallback. Complete, resource-bounded, and failed runs are separate states, and only complete runs contribute negative evidence.

4.4 Evidence Assembly

Detector-local tracing classifies nearby log, UI, storage, network, return, intent, and sharing operations for audit context. These observations remain distinct from configured IFDS sinks. Each query path records its source binding, source kind, sink endpoint, complete statement sequence, producing analysis, and Promise transfer derivations. A privacy-data path is linked to a detector occurrence only when project, normalized source location, and API identity agree. Framework-input and unlinked query paths remain separate records. This partial join prevents a project-level path from being attached to every privacy API in the same project.

5 Evaluation

We answer four questions: **RQ1** Can evidence-carrying source binding identify platform sources while rejecting same-name collisions? **RQ2** How does ARKPRISM compare with HapFlow on end-to-end source-to-sink classification? **RQ3** Which mechanisms account for the result, and are the resulting real-project paths semantically valid? **RQ4** Does the analysis scale, and what privacy evidence appears across a large ArkTS corpus? The source-first benchmarks answer RQ1 without using detector output as the oracle. HapBench provides the common end-to-end comparison for RQ2. Controlled ablations, asynchronous mutations, and manual path review answer RQ3. The corpus is used for prevalence and cost, not as an accuracy oracle.

5.1 Evaluation Protocol

Comparison setup. We compare ARKPRISM and HapFlow using ArkAnalyzer IR and an identical source-sink query over HapBench. HapBench is evaluated with its bundled API-17 SDK, while the real-project benchmarks and corpus use OpenHarmony API-20. Each experiment uses its designated SDK without fallback, and IFDS analyzes each project as a whole.

Identity benchmarks. *ArkSourceFirst60* fixes ten projects from each of six project-scale/configured-import strata before detector execution. An AST pass enumerates candidates under the fixed rule/import universe but assigns no labels. Reviewers first decide executable access, package ownership, and receiver origin from source, then consult IR only for aliases or lowered receiver forms. The resulting exact-location oracle contains 90 sensitive occurrences, 96 same-name or owner-incompatible negatives, and 44 confirmed-zero projects. *ArkIdentityBench* adds 32 positive and 32 matched-negative cases spanning imports, typed and constructed receivers, factories, properties, compound namespaces, and package migration.

The *Top-120 stress audit* complements the source-first oracle by concentrating projects with high API volume. Its 1,533 reviewed location rows canonicalize to 666 project-API keys with executable source evidence. ARKPRISM preserves all 666 confirmed keys, so this experiment measures confirmed-key coverage under a high-volume workload.

Flow benchmarks. *HapBench* comprises all 67 executable cases released with HapFlow: 53 positive and 14 negative. We preserve its published case oracle and additionally audit every canonical source-sink endpoint pair reported by ARKPRISM. *ArkAsyncBench* contains 48 source-first cases: 24 positives and 24 matched negatives over callback (T1), Promise-then (T4), and await (T5) carriers. Positives pass the configured carrier, a carrier property, or an alias to a sink; negatives pass an error, constant, rejection value, ignored result, or independent same-named local. *ArkPromiseBench* applies ten source-to-source boundary operators to 24 cases (seven positive and 17 negative): owner substitution, handler-position exchange, Promise alias mutation, payload-to-constant return replacement, rejection-operator insertion, chain extension and flattening, and callback registration without a trigger. Each mutant changes one semantic obligation while preserving the surrounding source and sink context.

Table 3. Identity evidence progression.

Configuration	TP	TN	FP	FN	Prec.	Rec.	Spec.	F1	Acc.	MCC
Lexical token	32	0	32	0	50.00%	100.00%	0.00%	66.67%	50.00%	0.000
Import-aware namespace	17	32	0	15	100.00%	53.13%	100.00%	69.39%	76.56%	0.601
Declared receiver	26	32	0	6	100.00%	81.25%	100.00%	89.66%	90.63%	0.827
Factory-aware receiver	28	32	0	4	100.00%	87.50%	100.00%	93.33%	93.75%	0.882
ARKPRISM	32	32	0	0	100.00%	100.00%	100.00%	100.00%	100.00%	1.000

Real-project path audit. A deterministic diversity sampler selects 120 paths from 47 projects: 90 privacy-data and 30 framework-input paths; 64 IFDS, 28 supplementary, and 28 dual-provenance paths; and 23 short, 70 medium, and 27 long paths. The sample includes every network and UI stratum. Source and IR review checks source identity, sink identity, explicit value dependence, reachability, and provenance. A path is accepted only when all five dimensions hold, and each decision retains its endpoints, complete statement path, supporting excerpts, and reason code.

Large corpus. The corpus contains 1,014 ArkTS projects analyzed under a common implementation, API-20 SDK, rule set, and resource policy. Its applications, libraries, and capability demonstrations expose diverse project roles and API usage patterns. Each project contributes one validated result; aggregation excludes incomplete analyses, unresolved endpoints, and invalid provenance.

Metrics. Identity occurrences match on project, normalized file, line/column, canonical package/namespace/member, and access kind. Flow benchmarks report precision, recall, specificity, balanced accuracy, F1, and MCC. Exact two-sided McNemar tests compare paired predictions; Wilson intervals describe HapBench rates and corpus proportions where sampling uncertainty is meaningful. Deterministic source-first, stress, mutation, and path audits report their exact reviewed ratios rather than population confidence intervals. Corpus statistics quantify project prevalence, concentration (Gini and HHI), Spearman associations, source scope, clone sensitivity, and runtime; the preceding oracles provide the accuracy evidence.

5.2 RQ1: Source-Binding Accuracy

Source-first accuracy. Within the fixed rule set and statically enumerable candidate universe, ARKPRISM reports exactly the 90 gold occurrences in ArkSourceFirst60: TP/FP/FN= 90/0/0, for 100% observed precision, recall, and F1. It also classifies all 16 positive and 44 zero projects correctly. Exact-location matching prevents one call from masking a missed occurrence elsewhere in the same file.

The gold set contains 44 executable namespace properties, 26 namespace calls, 11 typed receivers, four constructed receivers, three manager receivers, and two factory receivers. Thus, 20/90 occurrences require receiver evidence beyond the accepted line’s namespace syntax, and nearly half are property reads rather than calls. The 96 negatives stress the complementary boundary: application wrappers, UI controllers, collections, test drivers, and third-party methods reuse generic members such as create, on, request, and getData.

Evidence progression. Table 3 shows how the decision boundary changes. Lexical matching reaches every positive but accepts every matched collision. Import compatibility removes these collisions; receiver and factory evidence then recover indirect uses. The complete resolver adds constructed receivers, executable properties, compound aliases, and namespace-constrained migration, classifying all 64 cases correctly.

High-volume reproduction. ARKPRISM recovers all 666 manually confirmed project-API keys in the Top-120 stress audit (100% confirmed-key coverage), including 558 namespace-qualified, 102

Table 4. HapBench case-level classification.

Tool	TP	TN	FP	FN	Prec.	Rec.	Spec.	F1	Acc.	BAcc.	MCC
HapFlow [12]	51	10	4	2	92.73%	96.23%	71.43%	94.44%	91.04%	83.83%	0.717
ARKPRISM	50	14	0	3	100.00%	94.34%	100.00%	97.09%	95.52%	97.17%	0.881

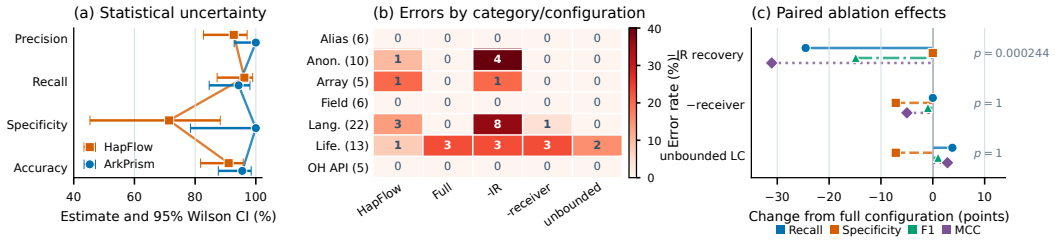


Fig. 5. HapBench uncertainty, category errors, and paired ablation effects.

member/receiver, five template-expression, and one compound-qualified key. This result demonstrates stable recognition across both dominant namespace calls and less common receiver, template, and compound forms under high API volume.

Answer to RQ1. On a source-first real-project oracle defined over the fixed rule and static-candidate universe, ARKPRISM resolves all 90 sensitive occurrences and rejects all 96 candidate collisions. The controlled benchmark attributes this result to the composition of package, receiver, factory, property, and IR evidence; the stress audit adds 102 confirmed member/receiver keys and shows that the evaluated detector configuration preserves all 666 previously confirmed high-volume keys.

5.3 RQ2: End-to-End Comparison with HapFlow

Table 4 reports all 67 HapBench cases. ARKPRISM classifies 64 correctly (50 TP, 14 TN), while HapFlow classifies 61 (51 TP, 10 TN). ARKPRISM rejects every negative case and retains 50/53 positives, yielding 100% precision and specificity, 97.09% F1, 97.17% balanced accuracy, and 0.881 MCC. Relative to HapFlow, specificity increases by 28.57 percentage points, F1 by 2.65 points, and balanced accuracy by 13.34 points.

The tools disagree on nine cases: ARKPRISM alone is correct on six and HapFlow alone on three (exact paired $p = 0.508$). The improvement comes from negative-case discrimination rather than broader source acceptance. Figure 5(a) shows Wilson uncertainty, and panel (b) localizes every error across seven categories and five configurations. ARKPRISM reaches 100% F1 in six categories and 85.71% in lifecycle cases.

The endpoint-pair audit tests precision inside positive cases. The 52 raw paths canonicalize to 51 source-sink pairs: 50 match the benchmark behavior, while one additional virtual target appears inside a positive case. With three missing positive endpoints, pair-level TP/FP/FN is 50/1/3, giving 98.04% precision, 94.34% recall, and 96.15% F1.

Why the classifications differ. The disagreements follow directly from the analyses' program models. In AnonymousClass2, the source lies in an anonymous instance initializer; retaining the lowered initializer makes the path visible. In Exceptions4, an exceptional successor carries the value into the handler. Conversely, ArrayIndexNoLeak requires distinct constant indices, and VirtualDispatch2/3 require the receiver-refined target set to remove infeasible callees. HapFlow misses the first two relations and retains the latter three, accounting for five of the six cases

Table 5. HapBench ablation results.

Configuration	TP	TN	FP	FN	Prec.	Rec.	Spec.	F1	Acc.	BAcc.	MCC
ARKPRISM (default)	50	14	0	3	100.00%	94.34%	100.00%	97.09%	95.52%	97.17%	0.881
–IR recovery	37	14	0	16	100.00%	69.81%	100.00%	82.22%	76.12%	84.91%	0.571
–receiver refinement	50	13	1	3	98.04%	94.34%	92.86%	96.15%	94.03%	93.60%	0.831
Unbounded lifecycle	52	13	1	1	98.11%	98.11%	92.86%	98.11%	97.01%	95.49%	0.910

classified correctly only by ARKPRISM. Bounded lifecycle ordering also rejects UnreachableFlow, where cycling from a later lifecycle phase creates a spurious path back to an earlier event.

The three published positives not emitted by ARKPRISM depend on stronger event assumptions: implicit execution of an overridden parent method, state shared across framework-created extension instances, or a new ability creation after a click callback. Appendix A.2 evaluates these assumptions under a stricter executed-order contract while preserving the published oracle as the primary result. **Answer to RQ2.** ARKPRISM combines 100% case-level precision and specificity with 97.09% F1 on the full executable suite. Its principal empirical advantage over HapFlow is precise rejection of negative cases; the endpoint audit retains 98.04% precision at the finer source–sink-pair unit.

5.4 RQ3: Mechanism Attribution and Path Validity

IR, receiver, and lifecycle mechanisms. Table 5 reports pre-specified ablations under the same query. Removing IR recovery loses 13 additional positives; the full configuration is exclusively correct on 13 cases and never exclusively wrong ($p = 0.000244$, significant after Bonferroni correction for four tests). Receiver refinement removes one infeasible virtual target without reducing recall.

The bounded default preserves all 14 negatives and attains 97.17% balanced accuracy, compared with 95.49% under unbounded lifecycle cycling. Removing the bound accepts two additional published positives but also recreates the benchmark’s explicit unreachable-flow negative. Because F1 omits true negatives, it ranks the unbounded setting higher on that narrower objective; balanced accuracy measures the positive and negative obligations together. Under the strict executed-order contract in Appendix A.2, bounded ARKPRISM classifies 67/67 cases, whereas unbounded cycling creates three cross-event false positives. The default therefore targets the audit objective: preserve negative-case discrimination unless an executable transition witness exists.

Continuation transfer. ArkAsyncBench isolates asynchronous carrier semantics with matched positive and negative cases. Both configurations in Table 6 disable the optional unresolved-IR fallback, so the comparison measures the IFDS transfer itself. Full IFDS classifies all 48 cases correctly. Removing only T4 preserves every T1 callback and T5 await result but misses all nine Promise-then positives, reducing recall from 100% to 62.50% and F1 from 100% to 76.92% ($p = 0.00390625$). All 24 negatives remain negative. ArkPromiseBench then separates payload propagation from edge executability. Full ARKPRISM classifies all 24 mutants correctly. Removing T4 retains all 17 negatives but misses all seven positives ($p = 0.015625$); admitting every function-typed SDK argument as an executable callback retains all positives but accepts a registered source callback for which no trigger exists, reducing precision to 87.50% and specificity to 94.12%.

The Promise suite instantiates the ownership, execution, and payload-dependence boundaries in Figure 4. User-defined or non-Promise then owners, rejection positions, catch/finally, unrelated or reassigned aliases, constant-return transforms, and registrations without a modeled trigger remain negative. Payload-dependent aliases, sequential chains, and Promise flattening remain positive; multi-hop paths carry both promise_then and promise_return derivations. The two ablations expose complementary obligations: T4 is necessary to retain payload identity across

Table 6. Asynchronous carrier isolation and Promise-boundary results.

(a) Balanced construct isolation							(b) Promise semantic mutations						
Construct	Pos.	Neg.	Full		-T4		TN	Correct classifications					
			TP	TP	TP	TP		Boundary	Cases	Pos.	Neg.	Full	-T4
T1 callback	9	9	9	9		0	9	Alias identity	4	1	3	4/4	4/4
T4 then	9	9	9	0		9	9	Callback execution	3	0	3	3/3	2/3
T5 await	6	6	6	6		0	6	Handler position	3	1	2	3/3	3/3
Total	24	24	24	15		9	24	Promise flattening	3	2	1	3/3	3/3
								Promise owner	2	0	2	2/2	2/2
								Rejection operator	2	0	2	2/2	2/2
								Sanitizer return	1	0	1	1/1	1/1
								Sequential chain	3	2	1	3/3	3/3
								Success handler	1	1	0	1/1	1/1
								Value dependence	2	0	2	2/2	2/2
								Total	24	7	17	24/24	23/24

Table 7. Manual semantic audit of 120 real-project paths.

Audited dimension	All	Privacy	Framework	IFDS	Supp.	Both	Rate
Source identity	120/120	90/90	30/30	64/64	28/28	28/28	100.00%
Sink identity	120/120	90/90	30/30	64/64	28/28	28/28	100.00%
Explicit value dependence	115/120	90/90	25/30	59/64	28/28	28/28	95.83%
Reachability	115/120	90/90	25/30	59/64	28/28	28/28	95.83%
Provenance label	120/120	90/90	30/30	64/64	28/28	28/28	100.00%
Complete path	115/120	90/90	25/30	59/64	28/28	28/28	95.83%

success continuations, while semantic edge selection is necessary to distinguish callback registration from callback execution.

Stratified semantic audit. Table 7 reports the 120-path source/IR review. Source identity, sink identity, and provenance are correct for every sampled path. All 90 sampled privacy-data paths are fully correct across the audited producer strata. Complete-path correctness in this stratified sample is 115/120 (95.83%).

The five rejected paths all belong to the optional framework-input model. Each starts from a lifecycle-written static URI field but reaches an unrelated component value: two input fields, two fixed validation messages, or an independent camera-picker result. This localization matters operationally: the core privacy-data analysis attains 90/90 in the audit, while the framework-input layer remains separately identifiable by source kind and provenance instead of silently entering the privacy-path claim. A corpus-wide change-impact audit isolates the receiver-witnessed platform-task contract: it adds one source-reviewed sensor path nested under a permission-completion callback while preserving the other 457 exact paths.

Answer to RQ3. IR recovery supplies the largest HapBench coverage gain, receiver refinement and lifecycle bounds preserve negative-case discrimination, and the continuation-aware IFDS transfer uniquely recovers nine Promise-then cases. Its Promise-owner, executable-edge, handler-position, alias, and return-dependence guards classify all 24 Promise mutation cases correctly. The stratified real-project audit confirms every sampled privacy-data path and 115/120 complete paths overall.

Table 8. Privacy evidence across 1,014 projects.

Evidence	Records	Projects	Project rate	Rec./positive proj.	Prod.	Test	Gen.
Privacy-API usages	2,339	353	34.81%	6.63	—	—	—
Detector-local sinks	1,755	224	22.09%	7.83	—	—	—
Privacy-data paths	258	31	3.06%	8.32	246	2	10
Framework-input paths	200	159	15.68%	1.26	45	155	0
All configured paths	458	183	18.05%	2.50	291	157	10

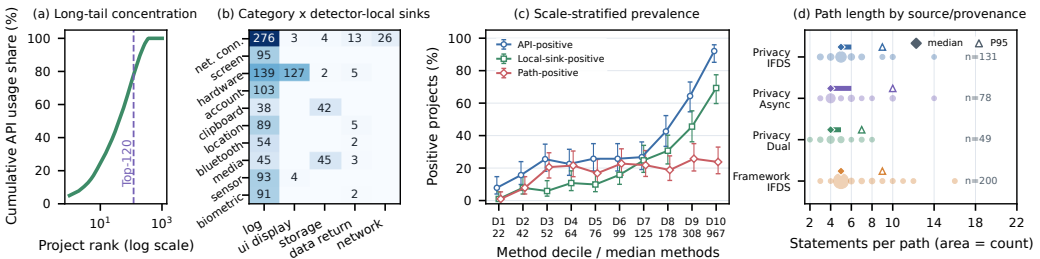


Fig. 6. Corpus evidence concentration, category-sink structure, scale-stratified prevalence, and provenance-conditioned path complexity.

5.5 RQ4: Large-Corpus Characterization

The corpus analysis covers 31,205 ArkTS/TypeScript files and 233,894 methods. Table 8 separates detector evidence, privacy-data paths, and framework-input paths. All 458 configured-query paths retain source and sink endpoints; source scope further distinguishes production, test, and generated code.

The scope split exposes two distinct corpus phenomena. Privacy-data paths overwhelmingly occur in production source (246/258), while framework-input paths are dominated by test abilities (155/200). We therefore use the former for privacy-flow claims and retain the latter as a separately labeled analysis of framework-provided values.

Concentration and structure. API evidence is long-tailed: 65.19% of projects have no recognized API, while the top 10, 50, and 120 projects contain 24.54%, 54.98%, and 77.85% of all occurrences. The API Gini coefficient is 0.858; HHI is 0.0105, equivalent to 95.6 equally sized contributing projects. Network connectivity (18.26%), screen identity (17.23%), hardware identity (15.09%), account data (8.08%), and clipboard data (5.09%) form the five largest categories.

Project size helps explain identity volume but not path formation. Method count correlates moderately with API occurrences ($\rho = 0.485$) and weakly with configured paths ($\rho = 0.160$); API occurrences correlate strongly with detector-local sinks ($\rho = 0.789$) and weakly with configured paths ($\rho = 0.184$). Figure 6(c) partitions projects into ten equally sized method-count strata and reports Wilson intervals. From the smallest to largest stratum, API-positive rates rise from 7.84% to 92.08% and detector-local-sink-positive rates from 0.98% to 69.31%. Path-positive rates instead remain between 18.81% and 25.74% across the four largest strata, separating codebase scale from configured-path formation.

Destinations and paths. Logging accounts for 1,441/1,755 detector-local sinks (82.11%), followed by UI display (136; 7.75%), storage (94; 5.36%), data return (58; 3.30%), and network (26; 1.48%). Call chains have median/P95 lengths of 1/4; configured paths have median/P95 lengths of 5/9 and a maximum of 16 statements. Provenance comprises 331 IFDS, 78 supplementary, and

49 dual paths. Figure 6(d) exposes their discrete structure: privacy IFDS, supplementary, and dual paths have medians of 5.0, 4.0, and 4.0 statements and P95 values of 9.0, 10.0, and 7.0, respectively; framework-input IFDS paths have a median of 5.0 and the longest tail (P95 9.0; maximum 16).

Integrity and cost. All 1,014 projects complete without IFDS batching, resource or callback truncation, timeout, or out-of-memory failure. The audit resolves all 3,337 trace endpoints, validates all 252 detector-to-path links, and finds 0 unresolved endpoints and 0 invalid links. The solver processes 3,006,526 IFDS edges in total; per-project median/P95/maximum is 739/10,811/187,278. Deduplication reduces 507 raw outcomes to 458 paths. Median runtime is 15.7 s, P95 is 21.6 s, and the 10,757-method outlier completes in 380.2 s.

Clone sensitivity. The token profile contains 31,998 source files and 21,430 unique fingerprints. It forms 9 exact-duplicate and 11 near-duplicate project clusters. Retaining one representative per exact cluster yields API/sink/path-positive rates of 35.18%/22.31%/18.19%; near-cluster representatives yield 35.05%/22.26%/18.23%. The maximum shift is 0.363 percentage points, showing that project-level rates are not driven by repeated copies of a small template set.

Answer to RQ4. ARKPRISM completes all 1,014 projects without resource fallbacks. Privacy-data paths are primarily production-source phenomena, identity and sink evidence are strongly concentrated, and clone-aware resampling leaves all project rates stable within 0.363 percentage points.

6 Discussion

6.1 Source Binding as an Analysis Interface

The central result is that API recognition and taint seeding should share an explicit interface. A canonical API identity alone cannot say which value enters the analysis, while an unqualified value seed cannot explain why the operation belongs to the platform source set. The binding $\langle \textit{identity}, \textit{carrier}, \textit{witness} \rangle$ connects these decisions. It also gives different analyses a common unit: the detector establishes the occurrence, IFDS propagates the carrier, and the evidence graph retains the witness.

This interface separates three audit questions. An operation can match a privacy rule without producing modeled data; a modeled carrier can reach a sink without establishing a policy violation; and a local sink can be relevant context without being the endpoint of that carrier. ArkPrism reports these relations separately, allowing an auditor to move from API evidence to data-flow evidence without inferring links from project-level co-occurrence.

6.2 Precision at Asynchronous Boundaries

ArkTS continuations require two kinds of evidence. Execution evidence establishes that a callback may run; carrier evidence establishes which value it receives. Treating a function type as both creates paths through callbacks that are merely stored or registered. Treating a Promise receiver as the payload loses the distinction between the container and its fulfilled value. ArkPrism instead re-roots an immutable source binding only at a witnessed success continuation and propagates a later Promise only when the handler return depends on that payload.

The same principle explains the lifecycle policy. A lifecycle edge supplies execution order, not data dependence. Bounded transitions preserve negative-case discrimination unless the program model provides an executable path between phases. This objective is reflected by balanced accuracy on HapBench and by the separate executed-order sensitivity analysis in Appendix A.2.

6.3 Transferability

The platform-specific inputs to ArkPrism are the export map, SDK ownership evidence, receiver factories, carrier contracts, and continuation triggers. The analysis interface and transfer discipline are not tied to ArkTS syntax. A typed event-driven platform can reuse the design by supplying adapters for API ownership and its callback or future abstraction. The reusable lesson is to preserve the source binding across frontend lowering and asynchronous execution rather than reconstructing source identity after propagation.

6.4 Scope and Interpretation

The source-first accuracy result is defined over the fixed rule set and statically enumerable candidate universe; the large corpus is used to characterize the analyzed collection. Source rules, SDK fingerprints, benchmark oracles, and run manifests are versioned in the replication package. Native C/C++ bridges, dynamically loaded code, runtime dispatch unavailable to ArkAnalyzer, server-side behavior, and consent state require complementary analyses such as HarmoBridge or runtime monitoring [14]. ArkPrism provides source-level may-flow evidence for the statically represented portion and leaves policy adjudication to the audit process.

7 Related Work

7.1 Mobile Privacy Sources and Sinks

Source and sink definitions determine the behaviors visible to a mobile taint analysis. Android work has derived these definitions from permissions, framework behavior, and API documentation, including Stowaway, PScout, SuSi, Axplorer, and FUSE [4, 5, 16, 33, 34]. Other systems connect code-level flows with permissions and privacy policies [2, 40, 44, 45]. These studies establish that API classification is a semantic input, not a neutral list of member names.

ArkPrism addresses a different but related step: binding a configured OpenHarmony source to its owner and data-carrying value in ArkTS. HapFlow resolves namespace–method entries to SDK declarations and uses the resulting signatures as IFDS sources [12]. This design is effective for source forms represented by the schema. ArkPrism retains signature resolution but adds witnesses for package re-exports, executable properties, factory-returned receivers, and compound IR aliases. Its contribution is therefore not a rejection of method signatures; it is an evidence-carrying binding between the signature-level identity and the concrete source carrier.

7.2 Implicit and Asynchronous Execution

Mobile analyses recover control flow that is absent from ordinary calls. Android systems model lifecycle, callbacks, and inter-component communication through dummy mains, framework summaries, callback mining, and ICC resolution [3, 10, 15, 20, 22, 24, 28, 29, 37]. Hybrid-app analyses additionally bridge native and script execution [8, 21, 41, 42]. This body of work shows that implicit edges must be constrained by platform semantics to avoid turning every registered callback into an execution path.

Promise and event analyses make a complementary distinction between scheduling and data transfer. Executes-before analysis models task-post order [31]; PromiseKeeper represents fulfillment, rejection, and chained reactions explicitly [1]; WEMINT applies API-specific asynchronous rules to mini-program ASTs [27]. ArkPrism brings these obligations into ArkTS IFDS. An execution witness admits the continuation edge, while a carrier fact identifies the fulfilled value. Promise ownership, handler position, alias identity, and payload-dependent return then determine whether that fact crosses a continuation.

7.3 OpenHarmony Analysis Infrastructure

ArkAnalyzer provides the source-level representation, control-flow graph, call graph, and analysis APIs used by ArkTS clients [11]. APAK extends this foundation with a context-sensitive pointer analysis and ArkTS-specific heap and closure modeling [43]. ArkPrism consumes these call-target and receiver facts when resolving source ownership. Neither framework defines a privacy source query by itself.

HapFlow is the closest end-to-end baseline because it accepts ArkTS projects and answers the same source-to-sink reachability query [12]. ArkPrism extends that analysis boundary with source binding, continuation-carrier transfer, receiver-refined dispatch, and evidence retention. HarmonyFlow instead analyzes compiled Ark Panda IR and focuses on pointer and call-edge recovery [38]. HScope also operates on OpenHarmony bytecode, using abstract interpretation to resolve indirect calls and inter-component communication for privacy-leak detection [26]. These bytecode analyses are complementary to ArkPrism's source-level goal: preserving ArkTS package, receiver, source-location, and carrier evidence before and during taint propagation.

Android analyzers provide broader methodological reference points. FlowDroid, Amandroid, DroidSafe, IccTA, HornDroid, COVERT, and PacDroid explore different combinations of lifecycle, context, points-to, and inter-component precision [3, 6, 9, 13, 17, 22, 39]. Their input languages and framework models differ from ArkTS, so we use their design principles rather than compare published numbers under incompatible queries.

7.4 Evaluation of Static Analyses

ReproDroid shows that analyzer comparisons require a common query, benchmark set, and execution environment [32]. TaintBench provides machine-readable real-world flow oracles with expert review [25]; mutation-based evaluation exposes unsound choices that fixed suites may not cover [7]. Our evaluation follows these principles by running the complete HapBench artifact under one query, retaining negative cases, auditing endpoint pairs, constructing source-first exact-location labels independently of detector output, and using semantic mutations to isolate Promise ownership, execution, and carrier transfer.

8 Conclusion

ArkPrism makes privacy-source binding explicit in ArkTS static analysis. It resolves a platform operation, its data-carrying value, and the witness for both decisions across package exports, manager receivers, executable properties, IR aliases, callbacks, Promise continuations, and await t. The resulting binding is preserved through IFDS and attached to each audit path.

The evaluation shows the value of this design at complementary levels. ArkPrism classifies all 186 source-first real-project candidates, increases HapBench specificity by 28.57 percentage points and F1 by 2.65 points over HapFlow, isolates nine Promise paths to continuation-carrier transfer, and classifies all 24 Promise semantic mutations. Manual review confirms all 90 sampled privacy-data paths, and the analysis completes 1,014 projects without resource fallback. These results establish evidence-carrying source binding as a practical foundation for precise OpenHarmony privacy analysis.

Data Availability

The replication package contains the ARKPRISM source and build scripts; API, source, sink, and package-migration rules; ArkSourceFirst60 and ArkIdentityBench; the Top-120 stress audit; HapBench, ArkAsyncBench, and ArkPromiseBench sources, oracles, and configuration outputs; the 120-path semantic review; and per-project results for the 1,014-project corpus. Run manifests bind

Table 9. Case-level error inventory for the main HapBench comparison.

Tool	Out.	Category	Case	Observed mechanism
HapFlow	FN	Anonymous	AnonymousClass2	Anonymous instance initializer omitted
HapFlow	FN	Language	Exceptions4	Exceptional successor/handler not recovered
HapFlow	FP	Array	ArrayIndexNoLeak	Distinct constant indices collapse
HapFlow	FP	Language	VirtualDispatch2	Infeasible receiver target retained
HapFlow	FP	Language	VirtualDispatch3	Infeasible receiver target retained
HapFlow	FP	Lifecycle	UnreachableFlow	Lifecycle cycle creates reachability
ARKPRISM	FN	Lifecycle	ActivityLifecycle4	Parent source method is bypassed by child override
ARKPRISM	FN	Lifecycle	BackupExtensionAbility	Oracle assumes state across extension instances
ARKPRISM	FN	Lifecycle	Button1	Oracle assumes click-written static state reaches later creation

Table 10. Sensitivity to strict lifecycle execution order.

Tool	TP	TN	FP	FN	Prec.	Rec.	Spec.	F1	Acc.	BAcc.	MCC
HapFlow [12]	48	10	7	2	87.27%	96.00%	58.82%	91.43%	86.57%	77.41%	0.622
ARKPRISM	50	17	0	0	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%	1.000

each result to analyzer and frontend commits, rule hashes, SDK fingerprints, feature flags, and resource policy. Machine-readable results regenerate every table and figure. Dataset redistribution follows each constituent project’s license; project identifiers and derived reports are supplied where source redistribution is restricted.

A Reproduction Details

A.1 HapBench Case-level Outcomes

Table 9 lists every error produced by either executable tool under the fixed 67-case oracle. Cases absent from the table are classified correctly by both tools. We retain the source-annotated oracle even where a more restrictive event or object-lifetime semantics would suggest a different label.

The two tools agree on 58 outcomes. Of the nine discordant cases, ARKPRISM alone is correct on six and HapFlow alone on three. The exact two-sided McNemar test is therefore based on the discordant pair (6, 3) and yields $p = 0.508$.

At the finer endpoint-pair unit, ARKPRISM’s 52 raw paths canonicalize to 51 source–sink pairs. Source annotations or direct source/IR review confirm 50; the additional pair is the NoLeak.log target at line 14 of VirtualDispatch1, whose allocation receives an empty constant. The retained audit records every pair, its source identity, endpoint lines, provenance, review mode, and decision; an integrity test reproduces TP/FP/FN= 50/1/3 and 98.04% precision.

A.2 Lifecycle-Oracle Sensitivity

The primary evaluation preserves all published/source-annotated HapBench labels. We additionally define a strict executed-order contract for the three disputed lifecycle cases: an override does not execute its parent body without an explicit super call; an instance field does not cross framework-created instances without a same-object witness; and a UI callback occurring after initial ability creation does not feed an earlier onCreate without an explicit later-creation edge. This contract relabels ActivityLifecycle4, BackupExtensionAbility, and Button1 as negative and changes no other case.

Table 11. HapBench uncertainty and category robustness.

Tool	Prec. 95% CI	Rec. 95% CI	Spec. 95% CI	Acc. 95% CI	BAcc.	MCC	Macro F1	Worst F1
HapFlow [12]	[82.7%, 97.1%]	[87.2%, 99.0%]	[45.4%, 88.3%]	[81.8%, 95.8%]	83.83%	0.717	95.73%	88.89%
ARKPRISM	[92.9%, 100.0%]	[84.6%, 98.1%]	[78.5%, 100.0%]	[87.6%, 98.5%]	97.17%	0.881	97.96%	85.71%
–IR recovery	[90.6%, 100.0%]	[56.5%, 80.5%]	[78.5%, 100.0%]	[64.7%, 84.7%]	84.91%	0.571	87.07%	66.67%
–receiver refinement	[89.7%, 99.7%]	[84.6%, 98.1%]	[68.5%, 98.7%]	[85.6%, 97.7%]	93.60%	0.831	97.57%	85.71%
Unbounded lifecycle	[90.1%, 99.7%]	[90.1%, 99.7%]	[68.5%, 98.7%]	[89.8%, 99.2%]	95.49%	0.910	98.81%	91.67%

Table 12. Per-category HapBench errors and F1.

Category	Cases	HapFlow err.	HapFlow F1	ARKPRISM err.	ARKPRISM F1	$\Delta F1$
Aliasing	6	0	100.00%	0	100.00%	0.00%
Anonymous constructs	10	1	93.33%	0	100.00%	+6.67%
Array-like structures	5	1	88.89%	0	100.00%	+11.11%
Field/object sensitivity	6	0	100.00%	0	100.00%	0.00%
General language features	22	3	91.89%	0	100.00%	+8.11%
Lifecycle modeling	13	1	96.00%	3	85.71%	–10.29%
OpenHarmony-specific APIs	5	0	100.00%	0	100.00%	0.00%
Total / macro	67	6	95.73%	3	97.96%	+2.23%

The tools then disagree on nine cases, all classified correctly only by ARKPRISM; the exact two-sided McNemar test gives $p = 0.00390625$. This secondary sensitivity contract does not replace the published oracle; it makes the object and event-order semantics responsible for the difference explicit. Both tables are derived from the same case predictions.

A.3 Uncertainty and Category Robustness

Table 11 reports Wilson intervals for the main configurations. The specificity intervals reflect HapBench’s 14 negative cases; balanced accuracy and MCC summarize the positive and negative obligations jointly.

A.4 Source-first Benchmark Protocol

ArkSourceFirst60 is selected before detector execution. Projects are partitioned by source-tree size and by whether their imports are compatible with the sensitive-API rules; deterministic ordering selects ten projects from each of the six strata. Candidate discovery collects the project-wide SDK-import universe and enumerates statically named calls and property reads compatible with that universe, including static element access, named-import calls, and simple callable aliases. It neither reads a ARKPRISM report nor assigns a gold label. The resulting recall metric is therefore defined over this fixed rule and static-candidate universe rather than over APIs absent from the policy or dynamically computed members.

Oracle construction proceeds in two passes. Source-only classification first accepts an occurrence only when its executable context establishes the configured package/namespace/member and access kind through an import, SDK receiver type, constructor, manager/factory definition, or executable property read; detector reports are not consulted. It rejects comments, strings, declarations, same-name application methods, and unrelated library/controller receivers. Source/IR adjudication then resolves ambiguous aliases and lowered receiver forms without adding candidates. The gold set contains 90 accepted occurrences; its final decision record preserves all 186 judgments, including

exact locations, evidence excerpts, reason codes, and 96 receiver- or owner-incompatible negatives. Validation checks every source location and canonical rule key and rejects duplicate occurrence identities. Evaluation joins detector and gold records on project, normalized file, exact line and column, canonical API, and access kind.

A.5 High-output Stress-audit Protocol

The 120-project stress audit labels project–namespace–member keys. A key is accepted only when executable source establishes the configured API through an imported namespace/package alias, a manager/helper definition chain, an SDK receiver type, or an executable property rule. Comments, strings, declarations, unrelated same-name methods, and ambiguous method-only matches are rejected. Each accepted key retains project, normalized package/namespace/member, access kind, source file/line, and a bounded source excerpt.

The construction set contains 1,533 reviewed location rows that canonicalize to 666 confirmed project–API keys. The companion integrity audit resolves every key against the delivered source tree and evaluated rules, verifies file/line bounds and snippets, and checks package/namespace/member compatibility. All 666 records pass. Because the set begins from an earlier output ranking, it is not an exhaustive occurrence oracle.

The detector configuration used for the corpus recovers all 666 confirmed keys in all 120 projects. It also reports 182 keys outside the reviewed set. The evaluator therefore defines *confirmed-key reproduction coverage* (666/666); unreviewed keys do not enter either side of this metric.

A.6 Real-project Path-audit Protocol

The path audit samples the complete set of 458 corpus paths. A deterministic sampler balances source kind, provenance, sink family, path-length bin, and project diversity, producing 120 records from 47 projects. The final review queue contains 90 privacy-data and 30 framework-input paths; 64 IFDS, 28 supplementary, and 28 dual-provenance paths; and all available network and UI strata. Each record retains endpoint locations, the complete statement path, source and IR excerpts, source identity, provenance, and a decision reason. Exact rebinding preserves 118 reviewed paths in the final result set. Two deterministic replacements receive fresh source review, and two exact paths whose derivation changes from supplementary to dual provenance are rechecked against their Promise declarations.

Each referenced source/IR path receives five Boolean decisions: source identity, sink identity, explicit value dependence, reachability consistency, and provenance label. A path is accepted only when all five hold. The final audit confirms source identity, sink identity, and provenance for 120/120 paths; explicit dependence, reachability, and complete-path correctness hold for 115/120. All 90 privacy-data paths are complete. The five rejected framework-input paths join a lifecycle-written static URI to unrelated input fields, fixed validation messages, or an independent camera-picker result.

A.7 Result Traceability

The replication package provides ArkSourceFirst60’s 186-item decision set and occurrence gold set; the 64-case ArkIdentityBench oracle; machine-readable sources, mutation operators, oracles, and configuration outputs for all 67 HapBench, 48 ArkAsyncBench, and 24 ArkPromiseBench cases; the Top-120 reproduction results; and the complete semantic-path audit. These records derive confusion matrices, construct/category metrics, endpoint-pair outcomes, paired tests, uncertainty intervals, and runtimes. Figure 5 and Figure 6 are generated from evaluator JSON rather than transcribed values.

Corpus tables and figures use one validated result per project under a common analyzer build, SDK, rule set, and resource policy. Aggregation checks project inventory, completion state, source and sink endpoints, provenance, and detector-to-path links. Every run manifest records analyzer and frontend commits, source/sink/rule hashes, SDK fingerprint, continuation and fallback flags, memory budget, and timeout policy. Companion JSON records every plotted value and its evaluation configuration.

References

- [1] Saba Alimadadi, Di Zhong, Magnus Madsen, and Frank Tip. 2018. Finding Broken Promises in Asynchronous JavaScript Programs. *Proceedings of the ACM on Programming Languages* 2, OOPSLA, Article 162 (2018), 26 pages. doi:10.1145/3276532
- [2] Benjamin Andow, Samin Yaseer Mahmud, Justin Whitaker, William Enck, Bradley Reaves, Kapil Singh, and Serge Egelman. 2020. Actions Speak Louder than Words: Entity-Sensitive Privacy Policy and Data Flow Analysis with PoliCheck. In *29th USENIX Security Symposium (USENIX Security)*. USENIX Association, Online, 985–1002.
- [3] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Oceau, and Patrick McDaniel. 2014. FlowDroid: precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. Association for Computing Machinery, New York, NY, USA, 259–269.
- [4] Kathy Wain Yee Au, Yi Fan Zhou, Zhen Huang, and David Lie. 2012. PScout: Analyzing the Android Permission Specification. In *Proceedings of the ACM Conference on Computer and Communications Security*. 217–228.
- [5] Michael Backes, Sven Bugiel, Erik Derr, Patrick McDaniel, Damien Oceau, and Sebastian Weisgerber. 2016. On Demystifying the Android Application Framework: Re-Visiting Android Permission Specification Analysis. In *Proceedings of the USENIX Security Symposium*. 1101–1118.
- [6] Hamid Bagheri, Alireza Sadeghi, Joshua Garcia, and Sam Malek. 2015. COVERT: Compositional Analysis of Android Inter-App Permission Leakage. In *Proceedings of the IEEE/ACM International Conference on Software Engineering*. 866–877.
- [7] Richard Bonett, Kaushal Kafle, Kevin Moran, Adwait Nadkarni, and Denys Poshyvanyk. 2018. Discovering Flaws in Security-Focused Static Analysis Tools for Android Using Systematic Mutation. In *Proceedings of the 27th USENIX Security Symposium*. USENIX Association, 1263–1280. <https://www.usenix.org/conference/usenixsecurity18/presentation/bonett>
- [8] Achim D. Brucker and Michael Herzberg. 2018. BabelView: Evaluating the Impact of Code Injection Attacks in Mobile Webviews. In *Proceedings of the International Symposium on Research in Attacks, Intrusions, and Defenses*. 25–46.
- [9] Stefano Calzavara, Ilya Grishchenko, and Matteo Maffei. 2016. HornDroid: Practical and Sound Static Analysis of Android Applications by SMT Solving. In *Proceedings of the IEEE European Symposium on Security and Privacy*. 47–62.
- [10] Yinzhi Cao, Yanick Fratantonio, Antonio Bianchi, Antonio Chung, Yanick Huang, David Lie, Giovanni Vigna, and Christopher Kruegel. 2015. EdgeMiner: Automatically Detecting Implicit Control Flow Transitions through the Android Framework. In *Proceedings of the Network and Distributed System Security Symposium*.
- [11] Haonan Chen, Daihang Chen, Yizhuo Yang, Lingyun Xu, Liang Gao, Mingyi Zhou, Chunming Hu, and Li Li. 2025. ArkAnalyzer: The Static Analysis Framework for OpenHarmony. In *Proceedings of the 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE SEIP)*. ACM.
- [12] Haonan Chen, Mingyi Zhou, Yanjie Zhao, and Li Li. 2026. HapFlow: The Taint Analysis Framework for OpenHarmony Apps. *ACM Transactions on Software Engineering and Methodology* (2026). doi:10.1145/3793863
- [13] Menglong Chen, Tian Tan, Minxue Pan, and Yue Li. 2025. PacDroid: A Pointer-Analysis-Centric Framework for Security Vulnerabilities in Android Apps. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*. 2803–2815. doi:10.1109/ICSE55347.2025.00208
- [14] Zhen Chen, Yi Li, Teng Wang, et al. 2025. HarmoBridge: Static Analysis of Cross-Language Data Flows in HarmonyOS Applications. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.
- [15] Erika Chin, Adrienne Porter Felt, Kate Greenwood, and David Wagner. 2011. Analyzing Inter-Application Communication in Android. In *Proceedings of the 9th International Conference on Mobile Systems, Applications, and Services*. 239–252.
- [16] Adrienne Porter Felt, Erika Chin, Steve Hanna, Dawn Song, and David Wagner. 2011. Android Permissions Demystified. In *Proceedings of the ACM Conference on Computer and Communications Security*. 627–638.
- [17] Michael I. Gordon, Deokhwan Kim, Jeff H. Perkins, Limei Gilham, Nguyen Nguyen, and Martin Rinard. 2015. Information Flow Analysis of Android Applications in DroidSafe. In *Proceedings of the Network and Distributed System Security Symposium*.
- [18] Huawei. 2021. HarmonyOS 2 Security Technical White Paper. <https://consumer.huawei.com/>

- [19] Huawei Developer. 2026. ArkTS: Dedicated Programming Language for the HarmonyOS Ecosystem. <https://developer.huawei.com/consumer/en/arkts/>
- [20] William Klieber, Lori Flynn, Amar Bhosale, Limin Jia, and Lujo Bauer. 2014. Android Taint Flow Analysis for App Sets. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on the State of the Art in Java Program Analysis*. 1–6.
- [21] Sungho Lee, Julian Dolby, and Sukyoung Ryu. 2016. HybriDroid: static analysis framework for Android hybrid applications. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. Association for Computing Machinery, New York, NY, USA, 250–261. doi:10.1145/2970276.2970368
- [22] Li Li, Alexandre Bartel, Tegawendé F. Bissyandé, Jacques Klein, Yves Le Traon, Steven Arzt, Siegfried Rasthofer, Eric Bodden, Damien Octeau, and Patrick McDaniel. 2015. IccTA: Detecting Inter-Component Privacy Leaks in Android Apps. In *Proceedings of the 37th International Conference on Software Engineering*. 280–291.
- [23] Li Li, Xiang Gao, Hailong Sun, Chunming Hu, Xiaoyu Sun, Haoyu Wang, Haipeng Cai, Ting Su, Xiapu Luo, Tegawendé F. Bissyandé, Jacques Klein, John Grundy, Tao Xie, Haibo Chen, and Huaimin Wang. 2025. Software Engineering for OpenHarmony: A Research Roadmap. *Comput. Surveys* 58, 2, Article 34 (2025), 34 pages. doi:10.1145/3720538
- [24] Long Lu, Zhichun Li, Zhenyu Wu, Wenke Lee, and Guofei Jiang. 2012. CHEX: Statically Vetting Android Apps for Component Hijacking Vulnerabilities. In *Proceedings of the ACM Conference on Computer and Communications Security*. 229–240.
- [25] Linghui Luo, Felix Pauck, Goran Piskachev, Manuel Benz, Ivan Pashchenko, Martin Mory, Eric Bodden, Ben Hermann, and Fabio Massacci. 2022. TaintBench: Automatic Real-World Malware Benchmarking of Android Taint Analyses. *Empirical Software Engineering* 27, 1 (2022), 16. doi:10.1007/s10664-021-10013-5
- [26] Aohan Mei, Guangliang Yang, Xinming Guo, Yi Wang, Fuan Gui, and Min Yang. 2026. Fine-Grained Privacy Leakage Detection in OpenHarmony Apps. In *Proceedings of the ACM on Software Engineering, ISSTA*. <https://conf.researchr.org/details/issta-2026/issta-2026-research-papers/205/Fine-Grained-Privacy-Leakage-Detection-in-OpenHarmony-Apps>
- [27] Shi Meng, Liu Wang, Shenao Wang, Kailong Wang, Xusheng Xiao, Guangdong Bai, and Haoyu Wang. 2023. WeMinT: Tainting Sensitive Data Leaks in WeChat Mini-Programs. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1403–1415. doi:10.1109/ASE56229.2023.00151
- [28] Damien Octeau, Somesh Jha, and Patrick McDaniel. 2013. Effective Inter-Component Communication Mapping in Android with Epicc: An Essential Step Towards Holistic Security Analysis. In *Proceedings of the 22nd USENIX Security Symposium*. 543–558.
- [29] Damien Octeau, Daniel Luchaup, Matthew Dering, Somesh Jha, and Patrick McDaniel. 2015. Composite Constant Propagation: Application to Android Inter-Component Communication Analysis. In *Proceedings of the 37th International Conference on Software Engineering*. 77–88.
- [30] OpenAtom Foundation. 2026. OpenHarmony: A Comprehensive Open Source Project for the All-scenario, Fully-connected, Intelligent Era. <https://www.openharmony.cn/>
- [31] Rekha Pai, Abhishek Uppar, Akshatha Shenoy, Pranshu Kushwaha, and Deepak D’Souza. 2022. Static Executes-Before Analysis for Event-Driven Programs. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. Association for Computing Machinery, 233–244. doi:10.1145/3540250.3549116
- [32] Felix Pauck, Eric Bodden, and Heike Wehrheim. 2018. Do Android Taint Analysis Tools Keep Their Promises?. In *Proceedings of the 26th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 331–341. doi:10.1145/3236024.3236029
- [33] Siegfried Rasthofer, Steven Arzt, and Eric Bodden. 2014. A Machine-learning Approach for Classifying and Categorizing Android Sources and Sinks. In *Proceedings of the Network and Distributed System Security Symposium*.
- [34] Talia Ravitch, Eric Creswick, Aaron Tomb, Adam Foltzer, Trevor Elliott, and Ledah Casburn. 2014. Multi-App Security Analysis with Fuse: Statically Detecting Android App Collusion. In *Proceedings of the ACM SIGPLAN International Workshop on the State of the Art in Java Program Analysis*. 4–9.
- [35] Thomas Reps, Susan Horwitz, and Mooly Sagiv. 1995. Precise Interprocedural Dataflow Analysis via Graph Reachability. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 49–61.
- [36] Mooly Sagiv, Thomas Reps, and Susan Horwitz. 1996. Precise Interprocedural Dataflow Analysis with Applications to Constant Propagation. *Theoretical Computer Science* 167, 1–2 (1996), 131–170.
- [37] Jordan Samhi, Alexandre Bartel, Tegawendé F. Bissyandé, and Jacques Klein. 2021. RAICC: Revealing Atypical Inter-Component Communication in Android Apps. *Empirical Software Engineering* 26, 6 (2021).
- [38] Yue Wang, Sen Chen, Jiaming Li, Wenjing Dang, Renchao Chai, Ziyue Shi, and Li Li. 2025. HarmonyFlow: A Static Analysis Framework for HarmonyOS Applications Based on Ark Panda IR. *Journal of Software* (2025), 1–19. doi:10.13328/j.cnki.jos.007477
- [39] Fengguo Wei, Sankardas Roy, Xinming Ou, and Robby. 2018. Amandroid: A Precise and General Inter-component Data Flow Analysis Framework for Security Vetting of Android Apps. *ACM Trans. Priv. Secur.* 21, 3 (2018), 14:1–14:32.

- [40] Meng Xia, Lu Gong, Yuan Lyu, Zhengwei Qi, and Xue Liu. 2015. Effective Real-time Android Application Auditing. In *Proceedings of the IEEE Symposium on Security and Privacy*. 899–914.
- [41] Guangliang Yang, Abner Mendoza, and Guofei Gu. 2018. Automated Generation of Event-Oriented Exploits in Android Hybrid Apps. In *Proceedings of the Network and Distributed System Security Symposium*.
- [42] Guangliang Yang, Abner Mendoza, Jialong Zhang, and Guofei Gu. 2017. BridgeScope: Precisely and Scalably Vetting JavaScript Bridge Security Issues in Android Hybrid Apps. In *Proceedings of the International Symposium on Research in Attacks, Intrusions, and Defenses*. 166–185.
- [43] Yizhuo Yang, Lingyun Xu, Mingyi Zhou, and Li Li. 2025. Context-Sensitive Pointer Analysis for ArkTS. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering, Industry Showcase*. <https://conf.researchr.org/details/ase-2025/ase-2025-industry-showcase/14/Context-Sensitive-Pointer-Analysis-for-ArkTS>
- [44] Zheming Yang, Min Yang, Yuan Zhang, Guofei Gu, Peng Ning, and X. Sean Wang. 2013. AppIntent: Analyzing Sensitive Data Transmission in Android for Privacy Leakage Detection. In *Proceedings of the ACM Conference on Computer and Communications Security*. 1043–1054.
- [45] Le Yu, Xiapu Luo, Jiachi Chen, Hao Zhou, Tao Zhang, Henry Chang, and Hareton K. N. Leung. 2021. PPChecker: Towards Assessing the Trustworthiness of Android Apps’ Privacy Policies. *IEEE Transactions on Software Engineering (TSE)* 47, 2 (2021), 221–242.